
CSI 2120 – The Resident Matching Service Go Edition

Submitted to
Dr. Robert Laganriere

Submitted by
Nassar Shakir, 300082867
Kobe Blackburn, 300235492

University of Ottawa
Department of Electrical Engineering and Computer Science
Ottawa, Ontario

Submission date
February 26, 2026

Instructions

There are two subdirectories in the zip folder in our submission. The first being “Concurrent” and the other “Single”. These refer of course to the concurrent and single-threaded solutions.

To run the program, the user must first compile all classes then run the command “go run McVitie-Wilson.go” in the terminal. Within each subdirectory contains the csv files for the large datasets and they will run without issue. Ensure that before attempting to run the Go program you are in the desired subdirectory.

Solution and Approach

The recursive offer function is called for each resident once within the main function. First the residents are sorted by id and then they are called on by the offer function.

```
258 func main() {
259
260     // residentPath := "residentSmall.csv"
261     // programPath := "programSmall.csv"
262
263     // residentPath := "residents4000.csv"
264     // programPath := "programs4000.csv"
265
266     residentPath := "residentsLarge.csv"
267     programPath := "programsLarge.csv"
268
269     residents, _ := ReadResidentsCSV(residentPath)
270     programs, _ := ReadProgramsCSV(programPath)
271
272     start := time.Now()
273
274     var ids []int
275     for id := range residents {
276         ids = append(ids, id)
277     }
278     sort.Ints(ids)
279
280     for _, id := range ids {
281
282         globalWG.Add(1) //remove for single-threaded version
283         //remove go statement for single-threaded version
284         go offer(id, residents, programs)
285         //offer(id, residents, programs)
286     }
287
288     globalWG.Wait() //remove for single-threaded version
289
290     end := time.Now()
291
292     printStableMatch(residents, programs, end.Sub(start))
293 }
294
```

The offer function proposes to programs potential residents. First the function checks to see if the resident has been matched in which case it immediately returns. If the resident has not been matched then the program matched are evaluated in order of the resident's ROL.

```
126 var globalWG sync.WaitGroup //remove for single-threaded version
127
128 func offer(rid int, residents map[int]*Resident, programs map[string]*Program) {
129     defer globalWG.Done() //remove for single-threaded version
130
131     r := residents[rid]
132
133     r.mu.Lock() //remove for single-threaded version
134     if r.matchedProgram != "" {
135         r.mu.Unlock() //remove for single-threaded version
136         return
137     }
138     r.mu.Unlock() //remove for single-threaded version
139
140     for _, pid := range r.rol {
141         if evaluate(rid, pid, residents, programs) {
142             return
143         }
144     }
145 }
```

The evaluate function determines whether a program accepts or rejects a resident, or accepts a resident by replacing a previously accepted student's position thereby rejecting the previous student in favour of the current resident. If the program is full, the least-preferred matched resident will be compared with the current resident. The least-preferred student is displaced and the new resident takes their spot if the new resident is more preferred. Otherwise the student is rejected.

```

147 func evaluate(rid int, pid string, residents map[int]*Resident, programs map[string]*Program) bool {
148
149     p := programs[pid]
150     r := residents[rid]
151
152     p.mu.Lock() //remove for single-threaded version
153     defer p.mu.Unlock() //remove for single-threaded version
154
155     rank, ok := p.rank[rid]
156     if !ok {
157         return false
158     }
159
160     if len(p.selectedResidents) < p.nPositions {
161         p.selectedResidents = append(p.selectedResidents, rid)
162         r.matchedProgram = pid
163         return true
164     }
165
166     leastIndex := 0
167     leastRank := p.rank[p.selectedResidents[0]]
168
169     for i, sid := range p.selectedResidents {
170         if p.rank[sid] > leastRank {
171             leastRank = p.rank[sid]
172             leastIndex = i
173         }
174     }
175
176     if rank < leastRank {
177
178         displacedID := p.selectedResidents[leastIndex]
179         p.selectedResidents[leastIndex] = rid
180
181         r.matchedProgram = pid
182
183         displaced := residents[displacedID]
184         displaced.matchedProgram = ""
185
186         globalWG.Add(1) //remove for single-threaded version
187         //remove go statement for single-threaded version
188         go offer(displacedID, residents, programs)
189         //offer(displacedID, residents, programs)
190
191         return true
192     }

```

Results

The results of the McVitie-Wilson stable matching algorithm are shown below. It is shown that the concurrent implementation is more than 2x faster than the single-threaded approach. They have the same number of unmatched residents and unfilled positions which confirms the correctness and stability of the program.

	Number Unmatched	Positions Available	Execution Time
Residents4000	334	39	34 ms
Residents Concurrent	13027	94	10 s
Residents Single	13027	94	22 s